



Figure 1: AJP connector and *mod_jk* in action

```
<Channel className="org.apache.catalina.tribes.group.
GroupChannel">
  <Membership className="org.apache.catalina.tribes.membership.
McastService"
                address="228.0.0.4"
                port="45564"
                frequency="500"
                dropTime="3000" />
  <Receiver className="org.apache.catalina.tribes.transport.
nio.NioReceiver"
            address="192.168.1.253"
            port="4000"
            autoBind="100"
            selectorTimeout="5000"
            maxThreads="6000" />
  <Sender className="org.apache.catalina.tribes.transport.
ReplicationTransmitter">
  <Transport className="org.apache.catalina.tribes.transport.
nio.PooledParallelSender" />
</Sender>
<Interceptor className="org.apache.catalina.tribes.group.
Interceptors.TcpFailureDetector" />
<Interceptor className="org.apache.catalina.tribes.group.
Interceptors.MessageDispatch15Interceptor" />
</Channel>
<Valve className="org.apache.catalina.ha.tcp.
ReplicationValve" filter="" />
<Valve className="org.apache.catalina.ha.session.
JvmRouteBinderValve" />
<ClusterListener className="org.apache.catalina.ha.session.
ClusterSessionListener" />
<ClusterListener className="org.apache.catalina.ha.session.
JvmRouteSessionIDBinderListener" />
</Cluster>
```

Here, 228.0.0.4 is a multicast address for cluster multicasts.

You can adjust the cluster heart beat multicast frequency, *dropTime*, *selectorTime* and *maxThread* variables to suit you.

Next, let's integrate Apache and Tomcat server instances to each other using the *mod_jk*. Download the latest source for the Tomcat connectors from the official Tomcat mirror (<http://tomcat.apache.org/download-connectors.cgi>). Build the source on a test server (as it is not advisable to build the code on the target server in production) and paste it to the appropriate location.

```
tar -zxvf tomcat-connectors-version-src.tar.gz
cd tomcat-connectors-version-src/native
./configure --with-apxs=/usr/sbin/apxs --enable-api-
compatibility
make
make install
```

If you are building the connector on a different machine, copy it and paste it to */usr/lib64/httpd/modules*; if not, no action is needed.

Now, enable the *mod_jk* module in the *LoadModule* section of *httpd.conf* and add an *include* directive for the *mod_jk.conf* file as follows:

```
LoadModule jk_module modules/mod_jk.so
Include conf/mod_jk.conf
```

Tomcat integration to Apache is now complete. In the last step, let's configure Apache to accept requests from the clients and forward them to Tomcat instances called *workers*. For that to happen, we need to create the *mod_jk.conf* file inside */etc/httpd/conf* and put this in:

```
JkWorkersFile conf.d/workers.properties
JkLogFile /var/log/httpd/mod_jk/mod_jk.log
JkLogLevel debugs
JkRequestLogFormat "%w %W %T"
SSLEngine on
SSLCertificateFile /etc/pki/tls/certs/server.crt
SSLCertificateKeyFile /etc/pki/tls/private/server.key
SSLCACertificateFile /etc/pki/CA/cacert/cacert.pem
JkMount /* LoadBalancerCluster
JkShmFile logs/jk.shm
```

Omit the SSL directives if you do not need an encrypted connection between your Apache Web server and the Tomcat application servers. There are two directives worth noting above, *JkWorkersFile* and *JkMount*. The *JkWorkersFile* tells Apache where to find the file with the information about the cluster configuration, i.e., information about individual Tomcat application servers. The *JkMount* option needs more understanding, as it will change the behaviour of the entire HALB cluster. It tells the Apache Web servers which requests to serve and which to forward to the workers. So writing */SP/**

Continued on page No. 80...